gtfs-ingest: Pipeline & Algorithms

From GTFS feeds, a stops CSV and OpenStreetMap to stations, networks and interchanges — Guadalajara Metropolitan Area

Víctor Rodríguez
September 16, 2026

A visual companion to `PIPELINE_GUIDE.md` · charts measured on the run `out_mibici_v2` (2 GTFS feeds + MiBici, OSM network walk, 300 m proximity)

What the pipeline does

Four phases, 24 steps — each command consumes what the previous one wrote

Phase 1

`ingest`

Validate every source and load a clean stop inventory into PostGIS

steps 1–5

Phase 2

`analyze-frequency`

Headways, deduplicated stations, connectivity and walking networks

steps 6–17

Phase 3

`detect-interchanges`
`conflate-osm`

Interchanges, OSM coordinate checks, side of street

steps 18–22

Exports

`export-neo4j`
`google-qa`

Graph database package and opt-in external cross-checks

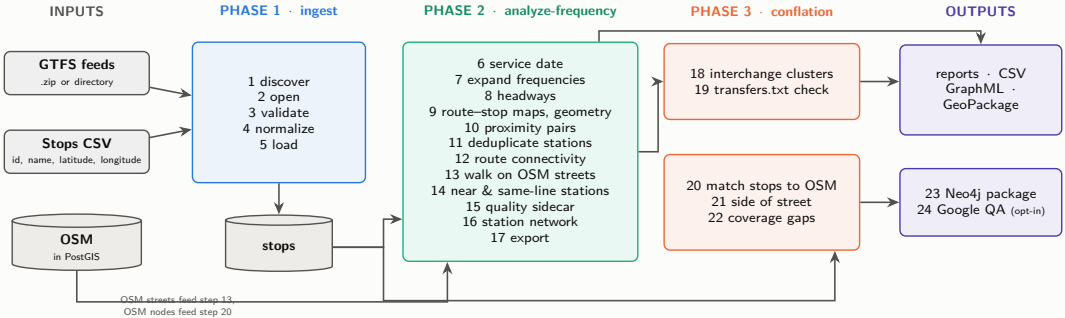
steps 23–24

14,349
stop records

8,877
physical stations

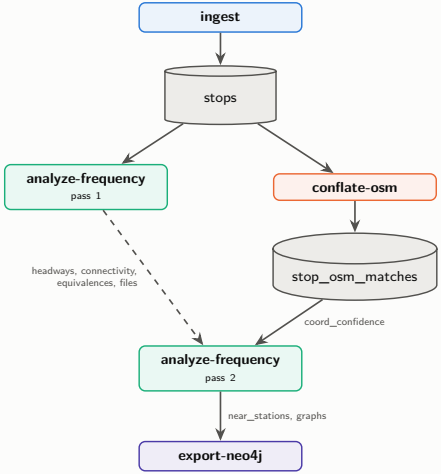
274
routes analysed

The pipeline at a glance



Run order and data lineage

Why `analyze-frequency` runs twice



- Each command reads what an earlier one wrote.
- The quality sidecar needs OSM verdicts that only `conflate-osm` writes ⇒ a second, cached `analyze-frequency` pass folds them in.
- `detect-interchanges` is independent and can run anywhere after ingest.

```
ingest               --feeds $FEEDS
analyze-frequency    --feeds $FEEDS \
    --network-walk --proximity-m 300
detect-interchanges --feeds $FEEDS
conflate-osm
analyze-frequency    ...    # pass 2
export-neo4j
```

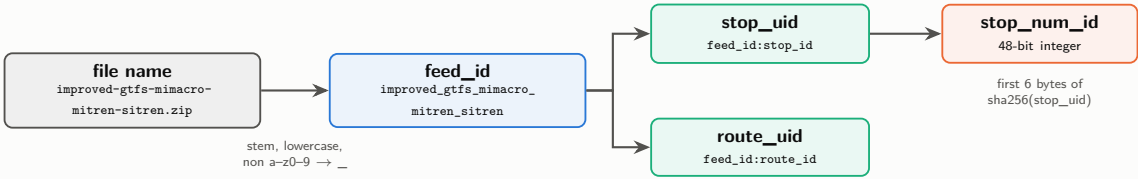
Concepts

Rules every step follows

Identity and provenance · coordinate systems · failure isolation
idempotent re-runs · records versus stations

Identity and provenance

Every identifier is qualified by the source it came from



Guarantees

- `feed_id` never contains `:`, so `stop_uid` splits back cleanly
- A CSV `S1` and a GTFS `S1` coexist — different `stop_uid`
- Same path ⇒ same id, run after run

Collisions

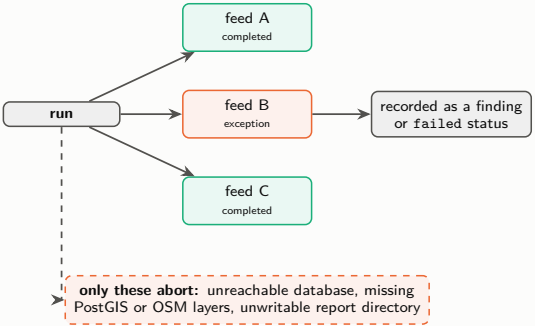
- Two sources with the same name: add 8 hex characters of `sha256(path)`
- `stop_num_id` collision odds $\approx n^2/2^{49}$ — about 10^{-4} at 150,000 stops; a unique index turns one into a load error, never silent corruption

Coordinate systems and failure isolation

Which coordinate system, where

Use	CRS
Data at rest: database and files	EPSG:4326
Planar maths: DBSCAN, local proximity, walk graph, side of street	UTM, auto-detected (32613)
Authoritative distances	PostGIS `geography`
OSM layers	read from the data (3857)

A broken feed never stops a run



Idempotency: how a re-run replaces data

Re-running any command is always safe — but the strategy depends on the table

Feed-scoped

DELETE rows of this feed, then insert — one transaction

`stops`
`route_stop_headways`
`route_headways`

Run-scoped derived

Truncate the whole table, then reload

`route_connections`
`stop_equivalences`
`interchanges`
`stop_osm_matches` · `side of street`

Budgeted verdicts

Upsert row by row — a partial re-check keeps earlier verdicts

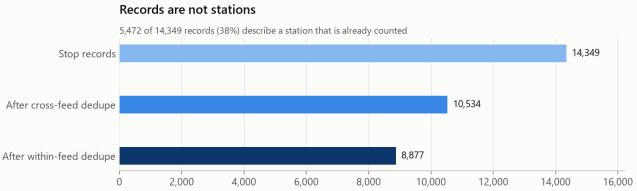
`google_qa_interchange`
`google_qa_stop`

Consequence

For run-scoped tables, pass **every** source you care about in one run: a run with two feeds leaves only those two feeds' connections in `route_connections`.

Records are not stations

Deduplication records equivalences — it never deletes a row



A raw `COUNT(*)` counts **records**. To count **physical stations**, resolve each record to its group with `station_dedup` from `near_stations_quality.csv`:

```
COUNT(DISTINCT COALESCE(
  NULLIF(station_dedup, ''),
  stop_uid))
```

Any density, coverage or accessibility figure on raw records is inflated by $\approx 1.6\times$.

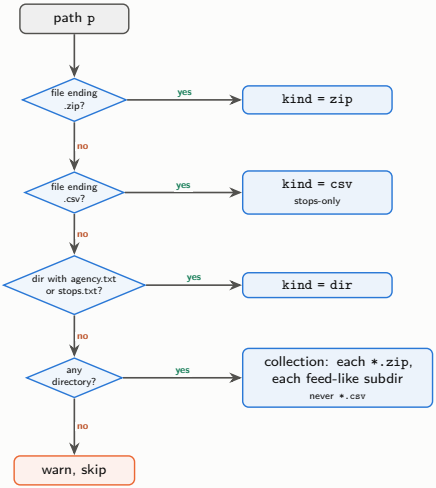
Phase 1 · Ingest

`ingest` — a clean, validated stop inventory in PostGIS

1 Discover sources **2** Open each source **3** Validate **4** Normalize stops **5** Load into PostGIS

STEP 1 Discover sources

Every path is classified once, and the kind is fixed at that moment



Output: a list of `FeedSource(feed_id, path, kind)`, de-duplicated by resolved path.

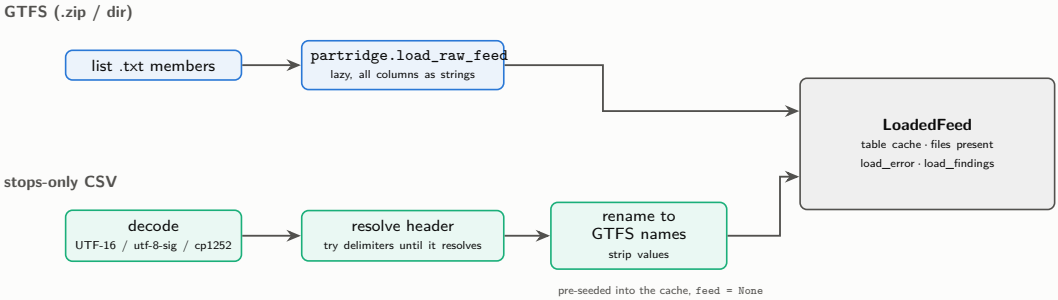
Why a CSV must be named explicitly

The pipeline's own `near_stations.csv` has `ID`, `stop_lat` and `stop_lon` — a *valid* stops CSV. Scanning folders for CSVs would silently re-ingest a previous run's output as a second copy of the network.

A directory named `foo.zip` stays a directory: the kind comes from the layout, not the suffix.

STEP 2 Open each source

Two ways in, one object out — every later step reads both the same way



Why read raw strings?

Validation must see the file *exactly as published*. Typing happens later, explicitly, in normalization.

Why pre-seed the CSV?

Table access consults the cache before the partridge object, so validators, the normalizer and Phase 2 read a CSV source with **no special cases**.

STEP 2 The stops-CSV reader: forgiving, never guessing

Header resolution

```
normalize(cell) =
    strip BOM and quotes
    -> NFKD, drop accents -> casefold
    -> non-alphanumeric runs -> "_"

for each delimiter candidate:
    parse (all values as strings)
    map normalize(cell) -> canonical
    if stop_id, stop_lat, stop_lon found:
        accept          # first match wins
# none resolved -> ERROR unresolved_csv_header
```

Examples: `Latitud`, `LATITUDE`, `lat` → `stop_lat`
`ESTACIÓN` → `stop_name`

Canonical	Accepted spellings
`stop_id`	id, stop_id, stopid, codigo, clave
`stop_name`	name, stop_name, stopname, nombre, estacion
`stop_lat`	latitude, lat, stop_lat, latitud
`stop_lon`	longitude, lon, lng, long, stop_lon, longitud

- **utf-8-sig first:** Spanish-locale Excel writes ;-separated UTF-8 with a BOM
- **Decimal commas** (20,6736) are flagged, never rewritten
- Unknown columns such as MiBici's `dpcapacity` are ignored
- The error names what was found, what is missing, and the spellings that would work

STEP 3 Validate

Every check runs — a crashing check becomes a finding, not a stopped run

	Check	Finding
ERROR	agency, stops, routes, trips, stop_times present	`missing_required_file`
ERROR	calendar.txt or calendar_dates.txt present	`missing_calendar`
ERROR	present files parse; required files non-empty	`unreadable_file`
ERROR	required columns per file	`missing_required_columns`
ERROR	stops: non-null, unique `stop_id`	`duplicate_stop_ids`
ERROR	stops: coordinates in WGS84 range	`invalid_coordinates`
WARN	`parent_station` references a known stop	`dangling_parent_station`
ERROR	trips→routes, stop_times→trips, stop_times→stops	`dangling_reference`
WARN	declared service ended before the analysis date	`feed_expired`
INFO	optional-file inventory	`optional_files`

Severity policy

ERROR spec violation or data loss
WARNING recoverable
INFO inventory

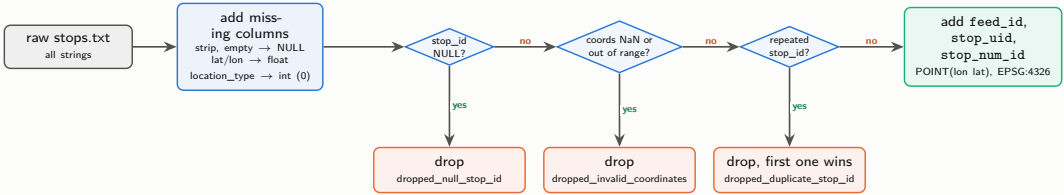
`--fail-on-error` exits 2 on any ERROR.

Stops-only sources

The required-files check becomes one INFO `stops_only_source`. Every other check still runs — a bad coordinate in a CSV raises the same finding as in a GTFS feed.

STEP 4 Normalize stops

Repair by exclusion only — a value is never guessed



Output columns

`stop_uid, stop_num_id, feed_id, stop_id, stop_name, stop_lat, stop_lon, location_type, parent_station, geometry`

Drop counts mirror the validator codes, so

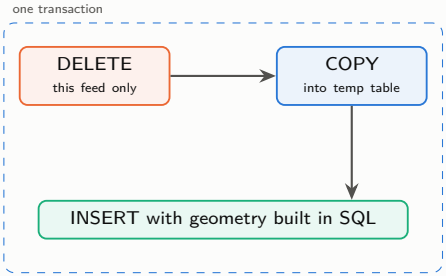
`source rows - dropped = normalized rows`

is always reconstructible from the report.

STEP 5 Load stops into PostGIS

Per-feed replace, bulk-copied, all inside one transaction

```
BEGIN;
DELETE FROM stops WHERE feed_id = :feed_id;
CREATE TEMP TABLE tmp_stops (...)
    ON COMMIT DROP;
COPY tmp_stops FROM STDIN;          -- bulk
INSERT INTO stops (..., geom)
SELECT ..., ST_SetSRID(
         ST_MakePoint(stop_lon, stop_lat), 4326)
FROM tmp_stops;
COMMIT;
```



- `COPY` fails? The same rows go through chunked `executemany` (1,000 per chunk) — same transaction
- Indexes: GiST on `geom`, `feed_id`, unique `stop_num_id`
- Other feeds are never touched

Phase 2 · Service analysis

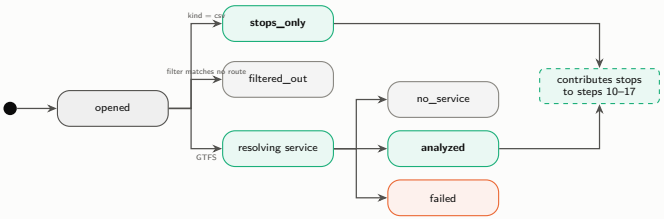
`analyze-frequency` — per feed, then across all feeds

Per feed: 6 service date 7 expand frequencies 8 headways 9 route–stop maps & geometry

Across feeds: 10 proximity 11 dedupe 12 connectivity 13 OSM walk 14 near stations 15 quality 16 station network 17 export

The life of one feed in Phase 2

Each feed runs inside its own failure boundary



Stops-only sources branch out first

No calendar and no trips, so every service-date path would fail. Instead they are normalized, loaded and marked `stops_only` with no service date — and still take part in steps 10–17.

- Exempt from `--transport-mode` / `--route`: nothing to match
- Excluded from `--common-date`: no calendar to share
- No service date, so they never make the run look incoherent

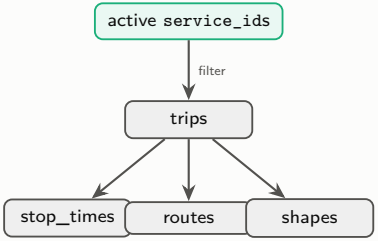
STEP 6 Resolve the service date

Headways depend on which trips actually run that day

Active services on a date

$$active(d) \;=\; \{\, s : \text{weekday flag set},\ start \le d \le end \,\} \;\cup\; \{\, s : \text{added on } d \,\} \;\setminus\; \{\, s : \text{removed on } d \,\}$$

Mode	Date chosen
`--service-date D`	exactly D
default	per feed, the date with the most active trips
`--common-date`	$\max\left(\bigcap_{\text{feeds}} \{\text{served dates}\}\right)$; exit 1 if empty
`--service-date-range`	every served date in the window, $\le$ 31 days

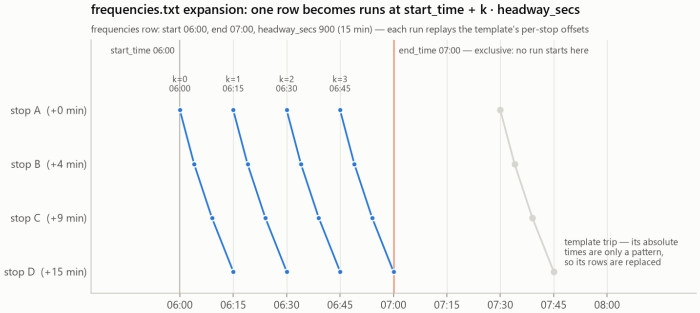


the filter cascades through foreign keys

Times are seconds since service-day midnight and may pass 24:00 (`25:10:00` = 90,600 s). Different days across feeds set `service_dates_coherent: false`.

STEP 7 Expand frequencies.txt

A frequency-defined trip is a pattern, not a departure

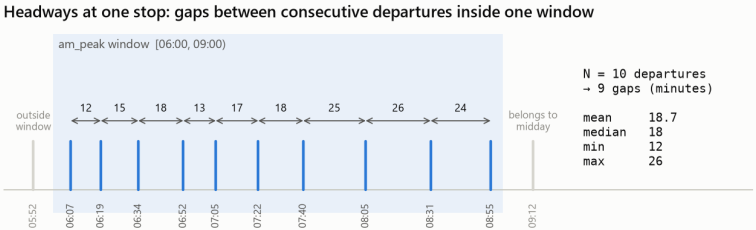



```
n_runs  = min(ceil((end - start) / headway), 1000)
offsets = departure(stop) - first departure
for k in 0 .. n_runs-1:
    emit (trip, stop, start + k*headway + offset)
drop duplicate rows; replace the template rows
```

- `end_time` is exclusive; invalid rows skipped with a warning
- Duplicates dropped: overlapping ranges would create 0-second gaps
- **181 of 184** and **970 of 989** trips are frequency-based here

STEP 8 Compute headways

The gap between consecutive departures, per stop, route, direction and window



Definition

Window $w = [start, end)$ · N departures $\Rightarrow N - 1$ gaps
`mean, median, min, max` of the gaps, in seconds
$N = 1$: headway undefined, statistics NULL

Details

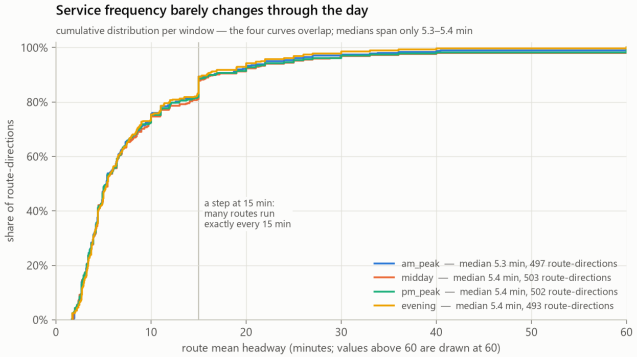
Default windows: am_peak 06–09, midday 09–15, pm_peak 15–19, evening 19–23
Blank `direction_id` → −1; directions never merged
Rows without a departure time are excluded

STEP 8 Route rollup — and what the data shows

Per route, direction and window

`mean_headway_s`	mean of stop-level means
`stop_count`	stops with a departure
`stops_with_headway`	stops with ≥ 2 departures
`trip_count`	max stop-level count

Unweighted: every stop counts equally.



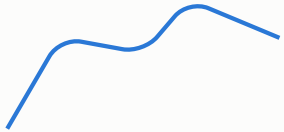
Read this carefully

Medians are 5.3–5.4 min in every window. The feeds define uniform frequencies all day — a property of the source data, **not** a finding about the city.

STEP 9 Route–stop maps and route geometry

Three views of the same routes, built from different files

route shapes



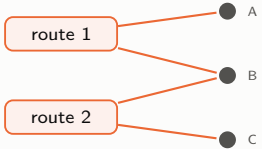
`shapes.txt` points in sequence —
the real drawn path, no stops, no time

route–stop links



consecutive stops from `stop_times` order,
`travel_min` = median across trips

route–stop map

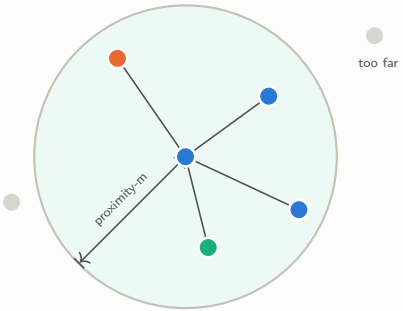


which routes serve which stops —
the source of every "served?" test

```
for each trip, ordered by numeric stop_sequence:
    seg_s = next departure - this departure          # negative -> NULL (broken sequence)
group by (route, direction, stop, next_stop):
    travel_min = round(median(seg_s) / 60, 1)       n_obs = count of non-NULL seg_s
```

STEP 10 Find proximity pairs

Every pair of stops closer than `--proximity-m`, across all feeds



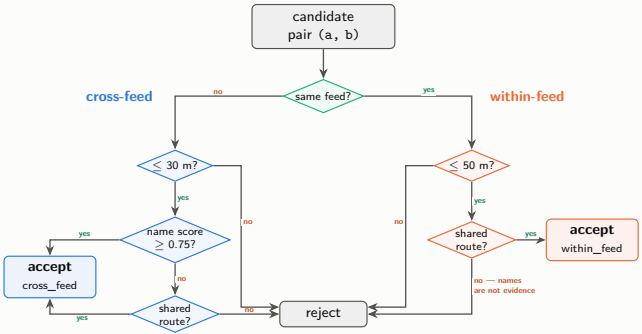
$$pairs = \{\, (a, b) : uid_a < uid_b,\ \ dist(a, b) \le r \,\}$$

Path	How
PostGIS	`ST_DWithin` on geography, GiST-indexed, on the `stops` table, filtered by feed
Local (`--dry-run`)	project to UTM, spatial-index join with `dwithin`

A trap dry runs cannot catch

The database path filters by feed id. Stops-only feeds must be in that list, or their rows match nothing — while the local path ignores the list and looks perfect.

STEP 11 Deduplicate stations: two rules, different evidence



Why names can't merge a within-feed pair

Two genuinely different stops of one feed can share a name — a long avenue, a plaza. Two stops of the **same route** within tens of metres are almost surely one station recorded twice: consecutive stops on a route sit ≈280 m apart.

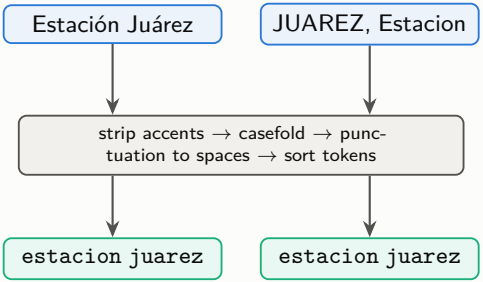
Safe for MiBici by construction

A stops CSV has no routes, so the within-feed rule can never fire for it.

Accepted pairs form a graph: each connected component is one station.

STEP 11 The evidence functions

Name similarity



SequenceMatcher ratio = 1.0

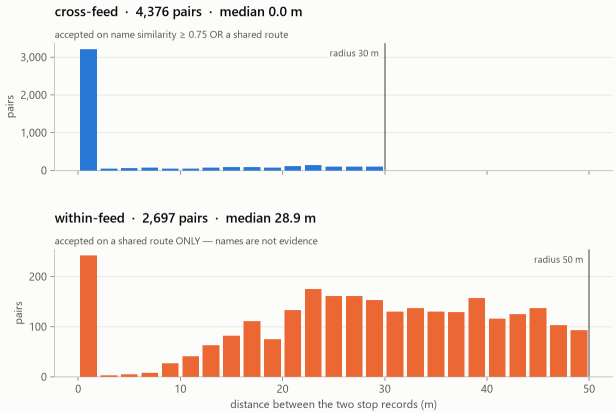
Shared routes and the audit score

```
routes(stop)  = { normalize(route_short_name)
                  of routes serving the stop }
shared_routes = | routes(a) & routes(b) |

score = 0.5 * (1 - dist / radius_that_admitted)
      + 0.5 * name_score        # audit only
```

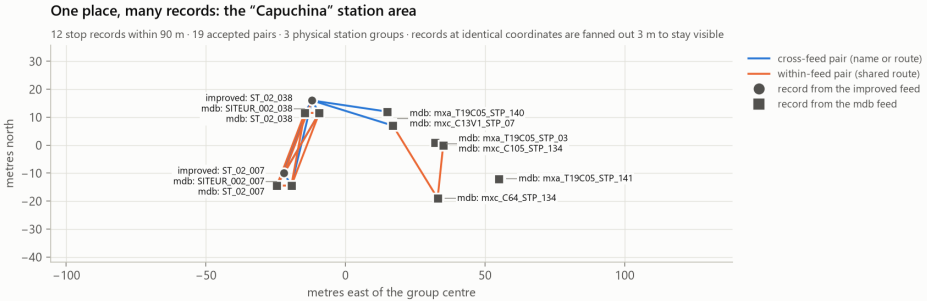
- Empty name ⇒ `name_score = 0`
- `score` is never a gate — it only helps review
- Every accepted pair records `match_kind`, so the two rules stay separately auditable

STEP 11 Dedupe measured: 7,073 accepted pairs



- **Cross-feed: 4,376 pairs, median 0.0 m.** Most are exact coordinate copies — the two feeds share a stop inventory.
- **Within-feed: 2,697 pairs, median 28.9 m.** Spread across the whole radius: one line published twice with slightly different platform positions.
- Merged groups span a median of 24 m and at most 103 m — **none above 150 m**, the guard against runaway chaining.

STEP 11 One real place, many records



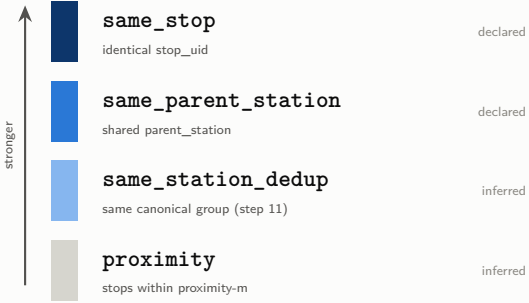
Both rules at once. Around one SiTren station, 12 records collapse into 3 groups: `ST_02_007` matches across feeds (blue); `SITEUR_*` and `ST_*` match inside one feed on shared routes (orange).

The trade-off, visible

`ST_02_007` and `ST_02_038` are the **two opposite-direction platforms** of one line — and they merge too. Right for counting stations; wrong for direction-level analysis.

STEP 12 Build route connectivity

Four ways two routes can meet — the strongest evidence names the edge



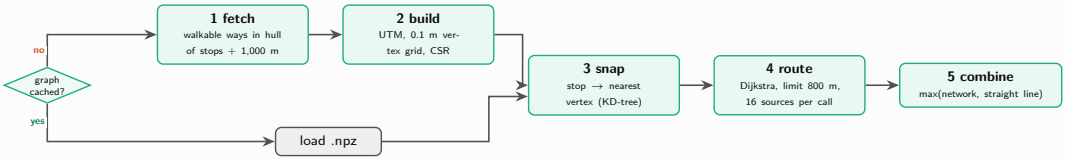
1. For each rule: `route_stops` ⋈ relation ⋈ `route_stops`
2. Drop self-pairs; order so `route_a < route_b`
3. Outer-merge the four sets on the route pair
4. Label with the **strongest** rule that produced it
5. `min_stop_distance_m` = 0 for `same_stop`, else closest proximity pair

Declared vs inferred

`same_station_dedup` stays separate from `same_parent_station`: one is inferred by the pipeline, the other declared by the operator.

STEP 13 Walk on real OSM streets

Straight-line distance understates walking — route it on the street network



Fetch & build

`highway` ways except motorway, motorway_link, proposed, construction. Endpoints snap to a 10 cm grid; ways connect only where they **share a coordinate** — bridges stay unconnected.

Snap & route

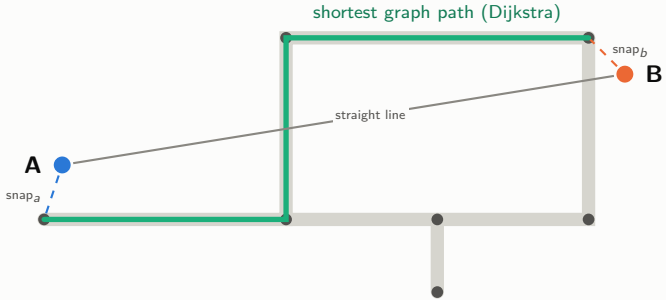
Nearest *vertex*, not nearest edge point. Snap > 200 m is labelled `unsnapped`. Each Dijkstra call allocates $16 \times 512{,}420$ floats ≈ 66 MB.

Cache key

Feed ids, hull buffer, OSM table and column, excluded highways, CRS — plus OSM row count, max `osm_id` and the stops' latest load time.

STEP 13 Inside the walk graph

How one pair of stops gets a walking distance



Combine

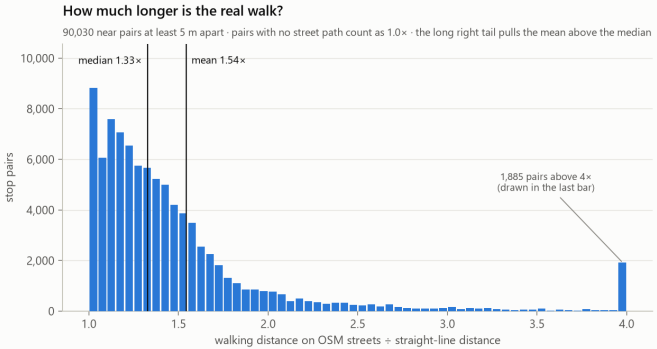
$network = path + snap_a + snap_b$

$distance = \max(network,\ straight\ line)$

no path within 800 m ⇒ keep the straight line, `walk_source = straight_line`

Applied **after** step 12 (connectivity stays geodesic) and **before** step 14 (walking minutes use real streets).

STEP 13 How much longer is the real walk?



1.33×

median walk / straight line

- 90,030 pairs at least 5 m apart
- Mean 1.54×: a long tail of big detours
- 1,885 pairs above 4×

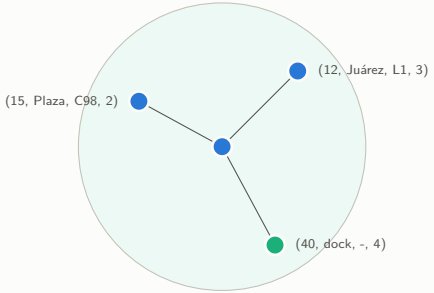
Use the median

The run log's "63 % detour" (1.63×) averages routed pairs only, including near-co-located stops whose tiny straight line inflates the ratio.

STEP 14 Near stations and same-line stations

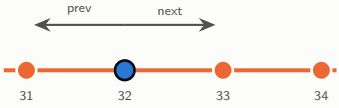
Two relations per stop, with deliberately different inclusion rules

near_stations — what can I walk to?



- Distance only, any line
- *minutes* = max(1, round(*metres*/83))
- Unserved neighbour listed with line –

same_line_stations — what's next?

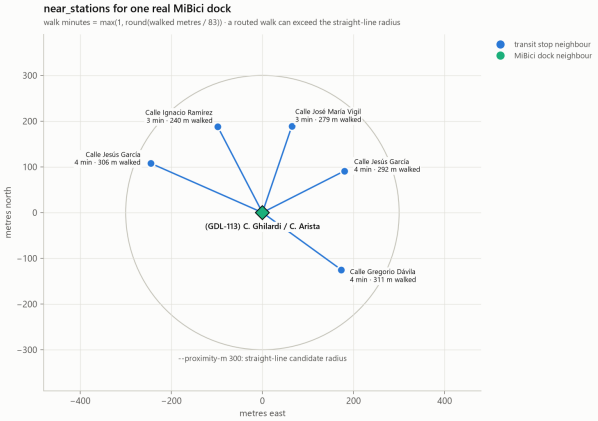


(31, Norte, L1, dir0, 1.5) (33, Centro, L1, dir0, 2.1)

- Sequence only, no distance limit
- Minutes = scheduled `travel_min`, else haversine at 25 km/h
- Previous *and* next stop, per line and direction

`ID` = 1..N in row order, referenced by every tuple; `stop_uid` is the cross-run key.

STEP 14 near_stations for one real MiBici dock



- Five transit neighbours, 3–4 minutes' walk
- Candidates are chosen by **straight line** (the circle)...
- ...but the reported distance is the **street walk** from step 13

So a walk can exceed the radius

Three neighbours are 292–311 m *walked* while sitting inside the 300 m straight-line circle.

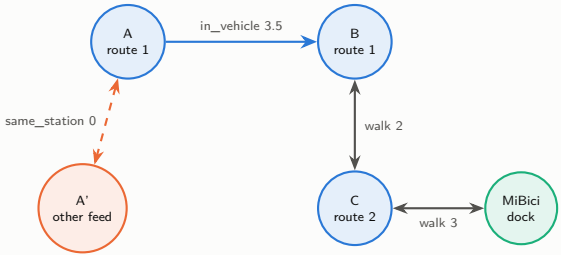
STEP 15 The quality sidecar: how each value was obtained

A measured 4 minutes and an estimated 4 minutes look identical — the sidecar tells them apart

Column	Values	Rule
`walk_source`	network, straight_line, mixed, unsnapped, none	label shared by the stop's neighbours; `mixed` if they disagree; `unsnapped` if snap $>$ 200 m
`travel_source`	observed, estimated, mixed, none	scheduled `travel_min` vs the 25 km/h fallback
`travel_n_obs`	integer	minimum trips behind the stop's medians — the weakest evidence
`coord_confidence`	0.812, needs_review, no_match, unavailable	from `stop_osm_matches` (step 20)
`station_dedup`	canonical_uid or empty	the stop's dedupe group — **use it to count stations**

Same rows and IDs as `near_stations.csv`; the main file's format never changes because of it.

STEP 16 The station network: a routable graph of places




```
nx.shortest_path(G, a, b, weight="time_min")
```

Edge	time_min	Built from
`in_vehicle`	`travel_min`	each route–stop link, one directed arc
`walk`	$\max(1, \mathrm{round}(m/83))$	each proximity pair, two arcs
`same_station`	0	each dedupe pair, two arcs

Riding, walking and transferring in one graph — and the only graph MiBici plugs into (walk edges, no in-vehicle edges).

Untimed segments leave some `in_vehicle` arcs without a time: filter or impute before shortest paths.

STEP 17 Export and persist

transit_analysis.gpkg

stops, near_links, route_shapes, route_stop_links, station_network

route_connectivity.graphml

routes and how they connect (step 12)

station_network.graphml

places, walking, riding, transfers (step 16)

near_stations.csv

walk and same-line relations (step 14)

near_stations_quality.csv

provenance of every value (step 15)

stop_equivalences.csv

dedupe pairs with match_kind (step 11)

route_*headways.csv

stop and route headways (step 8)

analysis_report.json + .md

dates, counts, connectivity, quality summary

PostGIS tables

headways per feed; connections and equivalences reloaded

Close any QGIS layer from the GeoPackage before re-running: Windows locks the open file and the export fails.

Phase 3 · Conflation

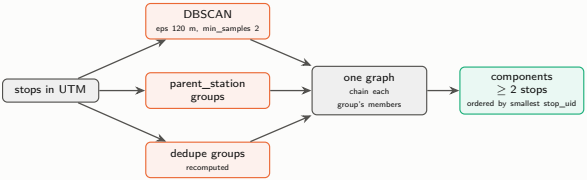
`detect-interchanges` and `conflate-osm`

18 Detect interchanges **19** Cross-check transfers.txt

20 Match stops to OSM transit nodes **21** Side of street **22** OSM coverage gaps

STEP 18 Detect interchanges

An interchange is infrastructure, not a schedule — whole feed, no service date



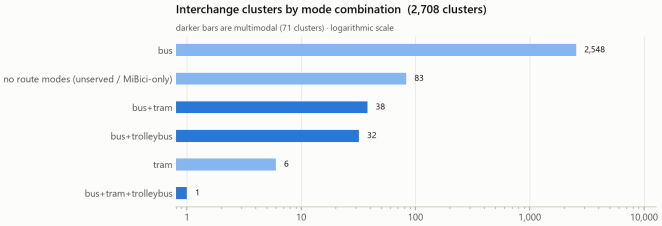

```
per cluster:
    modes                = distinct normalized route_type
    is_multimodal        = len(modes) > 1
    physical_stop_count = distinct canonical stations
    centroid, convex hull (EPSG:4326)
```

- A declared or aliased station is **never split**, even when its platforms are further apart than eps
- `route_type` normalized first: 700s → bus, 100s → rail, 400s → metro, 900s → tram
- `physical_stop_count = 1` flags a **phantom**: records of one station, not an interchange

Phase 2 vs Phase 3 dedupe

Phase 3 has whole-feed route evidence, so it finds **more** within-feed pairs: 2,752 versus 2,697. Expected, not a bug.

STEP 18 Interchange clusters by mode



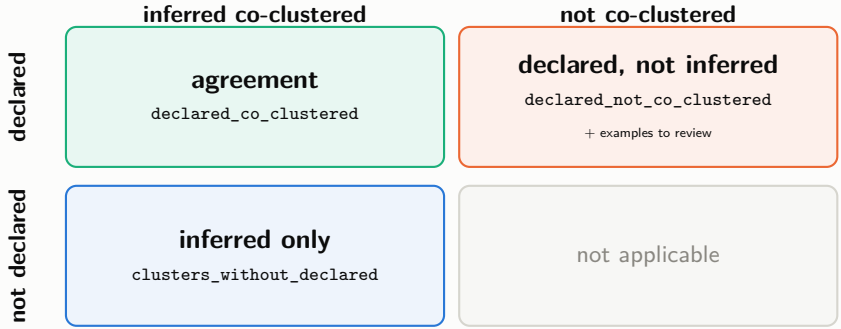
2,708
clusters

71
multimodal

809
single-station phantoms

STEP 19 Cross-check transfers.txt

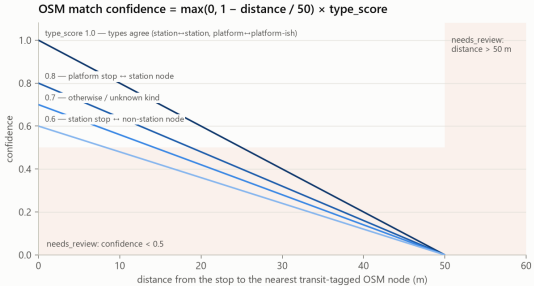
Do the clusters we inferred agree with the transfers the operator declared?



A feed without `transfers.txt` is listed in `feeds_without_transfers` — not an error.

STEP 20 Match stops to OSM transit nodes

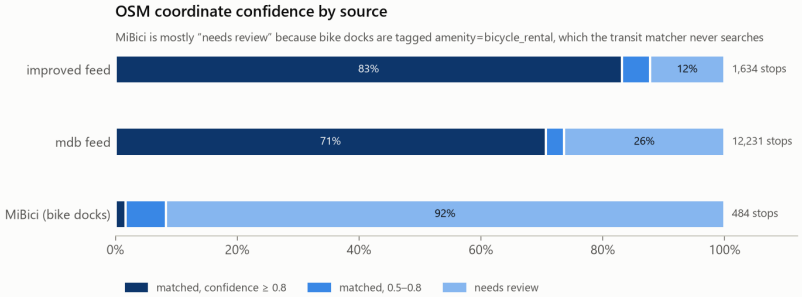
```sql
SELECT s.stop_uid, o.osm_id, o.kind, o.distance_m
FROM stops s
LEFT JOIN LATERAL (
  SELECT p.osm_id, ... AS kind,
    ST_Distance(s.geom::geography,
      ST_Transform(p.way,4326)::geography)
      AS distance_m
  FROM planet_osm_point p
  WHERE p.highway = 'bus_stop'
     OR p.public_transport IN
        ('platform','stop_position','station')
     OR p.railway IN ('station','halt','tram_stop')
  ORDER BY p.way <-> ST_Transform(s.geom, srid)
  LIMIT 1                          -- index KNN
) o ON TRUE;
```



- *confidence* $= \max(0,\ 1 - d/50) \times$ *type_score*
- `needs_review` $\Leftarrow$ no candidate, $d > 50$ m, or confidence < 0.5
- Schema validated first; fails fast listing every problem

STEP 20

OSM confidence by source



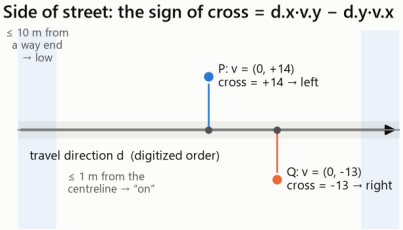
73.1%

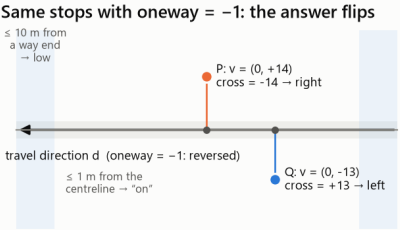
stops matched overall

MiBici's 92% "needs review" is a matcher-scope result

The query searches public-transport tags only. Bike docks are tagged `amenity=bicycle_rental` and are never searched, so a dock that sits exactly on its OSM node still comes back without a match. It is not evidence of bad coordinates.

STEP 21 Determine side of street





Rule

nearest highway centerline (KNN, $\leq 50\,\text{m}$) $\rightarrow$ nearest segment in UTM
d = direction *of travel* (reversed when `oneway = -1`)
v = stop minus its perpendicular foot
$cross = d_x v_y - d_y v_x$: > 0 left, < 0 right, $|v| \leq 1\,\text{m}$ on

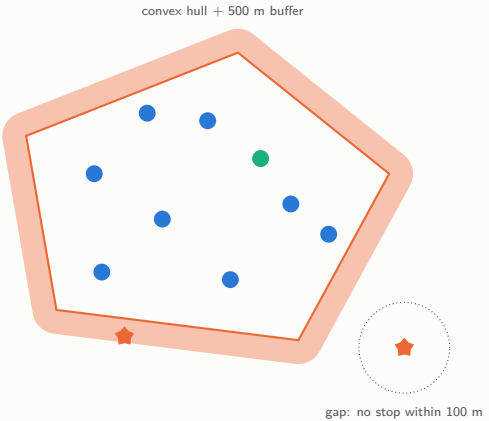
Low confidence when

side is `on` · foot within 10 m of a way end (an intersection) · $|v| < 2\,\text{m}$

Two-way streets: side is relative to OSM digitization. Rail stations snap to the nearest road; centre-median BRT platforms land in `on`.

STEP 22 Find OSM coverage gaps

The reverse question: what transit does OSM know that the feeds don't?



Algorithm

1. *area* $= \mathrm{buffer}(\mathrm{hull}(\text{all stops}), 500\,\mathrm{m})$
2. Each OSM transit node inside the area
3. Nearest stop by KNN, measured geodesically
4. Keep it if that stop is $> 100\,\mathrm{m}$ away
5. Sort worst gap first

A **review list, not a verdict**: a gap is either a hole in the feeds or noise in OSM.

34

OSM-only transit nodes

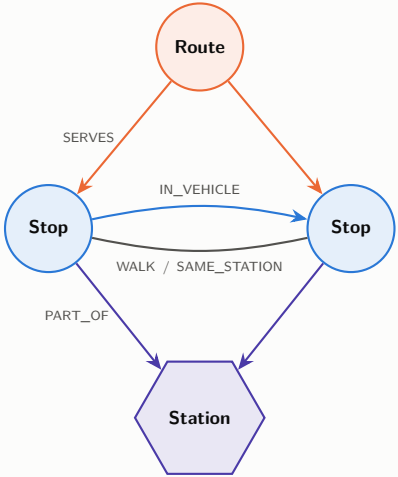
Exports

Share the results beyond the report directory

23 Neo4j load package — offline, from files only

24 Google QA and station locator — opt-in, budgeted, nothing stored but verdicts

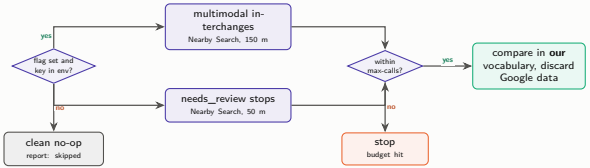
STEP 23 Export to Neo4j



- Reads **only files** from a report directory — works on any saved run
- `:Station` exists only for dedupe groups with more than one member
- `SERVES` is derived from `IN_VEHICLE` endpoints
- CSVs with `neo4j-admin` headers, plus an idempotent `load.cypher` (`IF NOT EXISTS`, `MERGE`, `IN TRANSACTIONS`)
- No data value is ever interpolated into Cypher

STEP 24 Google QA and station locator

Off by default — a flag *and* an API key in the environment are both required



Target	Verdicts
interchanges	CONFIRMED, PARTIAL, CONFLICT, NO_DATA
stops	STATION_FOUND, NOT_FOUND, NO_DATA

Licensing rails, enforced in code

Only the opaque `place_id` and the verdict are stored — a frozen column allowlist. No names, coordinates or lines, in the database, files or logs.

The station locator resolves station names and travel times, exports Excel, never writes to the database.

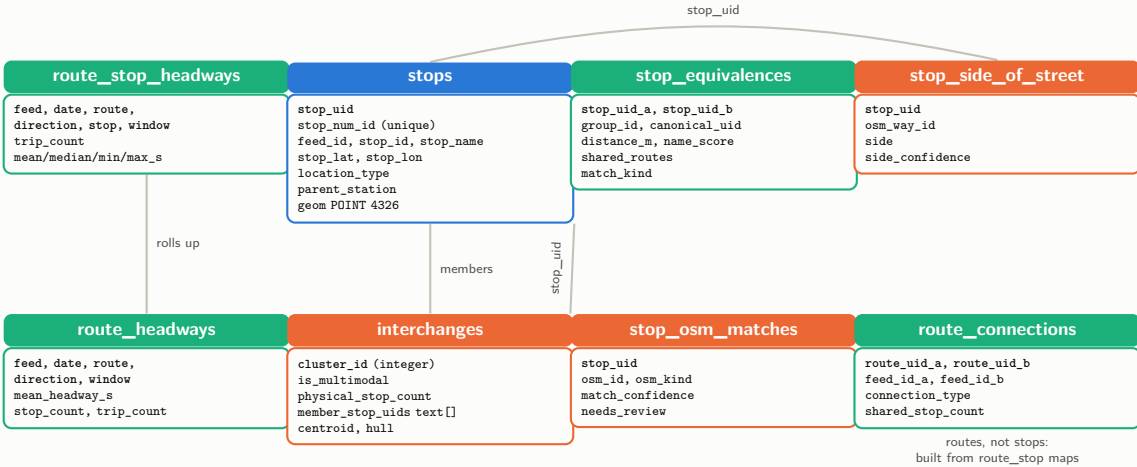
Results

The data model, the numbers, and how to read them

Data model · Guadalajara in numbers · choosing the proximity threshold
interpretation pitfalls · parameters · reproducing the run

The data model

Every analysis table keys back to `stops` — nothing is ever deleted from it



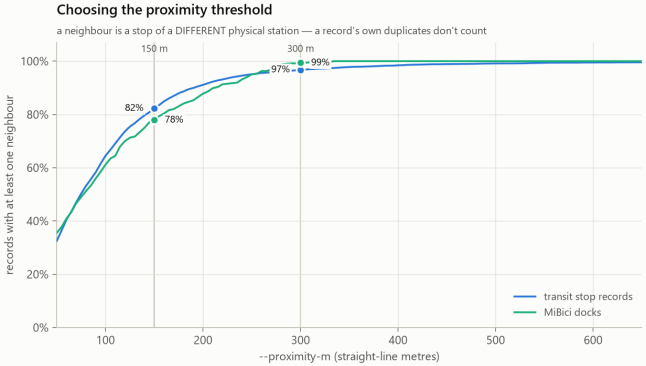
Guadalajara in numbers

Two GTFS feeds and the MiBici stops CSV · OSM network walk · 300 m proximity

14,349 stop records, 0 errors	**8,877** physical stations	**7,073** dedupe pairs	**9,112** route connections
102,460 stop-level headway rows	**512,420** walk-graph vertices	**1.33×** typical walk detour	**2,708** interchange clusters
73.1% stops matched to OSM	**67%** stops right of street (27% left)	**34** OSM coverage gaps	**484** MiBici docks, 0 merged

Service dates 2020-01-01 and 2024-01-01 (busiest per feed) — **not coherent**; pin one with `--service-date`.

Choosing the proximity threshold

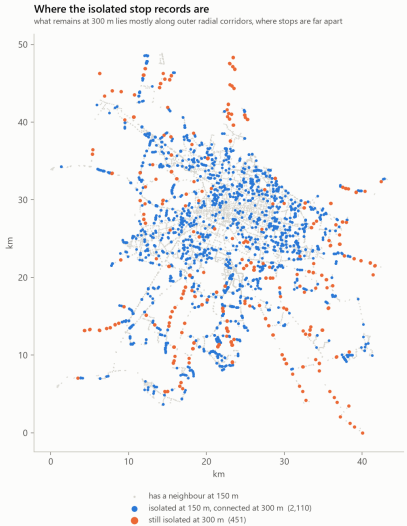


A neighbour is a stop of a **different** physical station — a record's own duplicates don't count.

	150 m	300 m
transit records with a neighbour	82%	97%
MiBici docks with a neighbour	78%	99%
isolated stations	1,236	206
station-network components	256	55
route connection edges	7,943	9,112

300 m ≈ a 3–4 minute walk.

Where the isolated stop records are



- **2,110** records were isolated at 150 m and connect at 300 m — mostly inside the urban area
- **451** remain isolated — mostly along outer radial corridors, where consecutive stops really are far apart

A threshold changes more than neighbour lists

`--proximity-m` also creates route `proximity` edges: 2,512 → 3,681. Compare connectivity figures only between runs with the same threshold.

Interpretation pitfalls

1. **Records are not stations.** Count with `COALESCE(station_dedup, stop_uid)`.
2. **Opposite platforms merge** under within-feed dedupe — fine for station counts, not for direction-level work.
3. **Check** `service_dates_coherent` before comparing feeds.
4. `direction_id` **is unreliable** here: some trunk lines use 0 for both directions.
5. **Empty** `system` has several causes — identify stops-only sources by `feed_id`.
6. **Headways don't vary by time of day** because the source data doesn't.
7. **Use the median detour** (1.33×), not the mean.
8. `needs_review` **for bike docks** is matcher scope, not bad coordinates.
9. **– in a near-stations tuple** marks a neighbour no line serves.
10. **Phase 2 and Phase 3 dedupe counts differ** by design.
11. `--proximity-m` **changes route edges**, not only neighbour lists.

Parameters that shape the results

Parameter	Default	Step	Controls	Parameter	Default	Step	Controls
`--service-date`	busiest	6	day analysed	`--network-walk`	off	13	OSM streets
`--common-date`	off	6	one shared day	`--eps`	120	18	DBSCAN radius
`--window`	4	8	headway windows	`--min-samples`	2	18	DBSCAN core
`--transport-mode`	all	6–9	routes kept	`--match-threshold-m`	50	20	OSM match
`--route`	all	6–9	routes kept	`--street-threshold-m`	50	21	centerline
`--proximity-m`	150	10–16	“near”	`--coverage-radius-m`	100	22	gap radius
`--dedupe-distance-m`	30	11	cross-feed radius	`--coverage-buffer-m`	500	22	service area
`--dedupe-name-threshold`	0.75	11	name similarity	`--max-calls`	200	24	Google budget
`--dedupe-within-feed-distance-m`	50	11	within-feed radius				

Fixed constants

walking 83 m/min · vehicle fallback 25 km/h · vertex grid 0.1 m · Dijkstra limit 800 m · snap label 200 m · at most 1,000 runs per frequency row

Reproduce this run

```
FEEDS="improved-gtfs-mimacro-mitren-sitren.zip
       mdb-2366-202502241922.zip mibici.csv"

python -m gtfs_ingest.cli ingest --feeds $FEEDS \
    --report ./out_mibici_v2
python -m gtfs_ingest.cli analyze-frequency \
    --feeds $FEEDS --report ./out_mibici_v2 \
    --network-walk --proximity-m 300
python -m gtfs_ingest.cli detect-interchanges \
    --feeds $FEEDS --report ./out_mibici_v2
python -m gtfs_ingest.cli conflate-osm \
    --report ./out_mibici_v2
python -m gtfs_ingest.cli analyze-frequency ...   # pass 2
python -m gtfs_ingest.cli export-neo4j \
    --report ./out_mibici_v2

python docs/figures/make_figures.py   # every chart here
```

Go deeper

- `PIPELINE_GUIDE.md` — every step, algorithm and pitfall
- `WORKFLOW.md` — the data, by concern
- `README.md` — install and operate
- `docs/` — Sphinx site with the API reference

Database URL from `DATABASE_URL` or `--db-url`. Add `--dry-run` to skip writes.